EXECUTION COST AND RECALL BUDGETS

Conditional hybrid retrieval

A BM25-first cascade decides when neural retrieval is worth its query cost. This study measures the gains after fixing an expensive lexical first stage.

20.21x

Combined TREC-COVID speedup over our earlier fusion baseline. Includes 16.36x from indexing and 1.24x additional routing; primary retains 96.68% of indexed fusion Recall@100.

7.91x

Combined speedup on the fixed SQuAD sample over our earlier fusion baseline. The routing-only gain is 1.59x over indexed fusion; r90 retains 99.22% on 256 questions.

These cross-run speedups compare stages of our Python implementation. They are not measurements against the original Elasticsearch / ELSER system. The results suggest a direction to test there, but do not establish that the same gains will transfer.

01 / CHOSEN PAPER

The paper that motivates this project

Sawarkar, Mangal and Solanki (2024) study keyword, dense and learned-sparse retrieval with hybrid field queries, including Elasticsearch and ELSER. Their work motivates combining complementary signals to supply better context.

The paper reports retrieval quality and generated-answer quality. Our extension investigates the query-time cost of running every retrieval branch.

Published reference values

TREC-COVID nDCG@10: **0.87**.
SQuAD answer EM / F1: **57.63 / 68.40**.

These values belong to the original paper. They are contextual references, not quality scores reproduced by our pipeline.

Our completed overlap

All 50 TREC-COVID queries and a declared SQuAD retrieval setup. NFCorpus supplies training/development evidence, while SCIDOCS supplies frozen transfer evidence.

The paper combines keyword, dense and learned-sparse retrieval to improve retrieved context. Our question is whether every query needs all three branches.

Source: Blended RAG, arXiv:2404.07220v2, sections IV–VI. The paper also discusses NQ, HotpotQA and metadata-poor CoQA; our current release does not claim completed evaluations on those datasets.

Same idea, different system

We reproduce the paper's use of complementary retrieval signals and field-aware BM25. Our implementation substitutes Python, MiniLM, SPLADE and reciprocal rank fusion (RRF) for Elasticsearch and ELSER, and adds a BM25-first gate. This is an adaptation of the paper's idea, not an exact reproduction of its system.

Original setting

Elasticsearch / ELSER, field-query combinations, top-k retrieval accuracy, nDCG and generated-answer EM/F1.

Current setting

MiniLM / SPLADE / RRF, a frozen Ridge gate, Recall@100, recall retention, nDCG@10, MAP@10 and measured query latency.

What “96.68% retained” means

Recall retained = mean method Recall@100 ÷ mean fusion Recall@100

It is a relative quality ratio. For TREC, fusion recall is 0.12250 and primary cascade recall is 0.11844. Their ratio is 96.68%. It does not mean recovering 96.68% of every query’s relevant documents or matching that fraction of fusion’s document IDs.

How the measurements differ

Hit accuracy asks whether at least one relevant document appears. Recall counts the fraction recovered. nDCG assesses the ordering and relevance of the top results. EM/F1 scores generated answers. Different models and relevance-label constructions add further comparison limits; this study does not isolate their individual causal effects.

Our SQuAD corpus contains 20,958 unique title/paragraph pairs from official train and dev data. The annotated parent paragraph is relevant for each dev question. Other answer-bearing paragraphs remain unjudged.

The initial cascade had little time to save

These are baseline results from our Python adaptation on the same two datasets used by the original paper. This overlap lets us compare our own implementation stages on shared data. It does not reproduce the original system or its published scores.

Dataset and scope	Queries	r90 recall retained	Fusion to cascade mean	Time saved
TREC-COVID	50	84.53%	1493.23 to 1440.00 ms	3.56%
SQuAD dev retrieval	10,570	99.38%	148.26 to 139.19 ms	6.12%

SQuAD timing has one complete pass. Its corpus contains 20,958 unique title/paragraph pairs from the official train and dev data. Each dev question uses its annotated parent paragraph as the relevance label.

Why our numbers differ from the paper

The systems do not share an engine or models: the paper uses Elasticsearch and ELSER, while ours uses Python BM25, MiniLM, SPLADE and RRF. The reported measures also differ. Their retrieval hit rate asks whether any relevant item appears near the top; our Recall@100 counts the share of judged relevant documents found. Their SQuAD EM/F1 scores generated answers, which we did not evaluate. The SQuAD document unit and relevance labels also differ. These are comparison limits; our experiments do not isolate the effect of each choice.

The initial TREC run exposed a runtime bottleneck: BM25 took 1410.14 ms of the 1493.23 ms fusion total. That left little time for a gate to save by skipping neural retrieval.

Transfer evidence

NFCorpus development r92 retained 94.74% and saved 23.00% mean time. Frozen SCIDOCS r92 retained 90.01% and saved 5.53%, with an interval crossing below 90%. The targets did not transfer consistently.

04 / IMPLEMENTATION

First fix the lexical bottleneck

A gate can return BM25 results for some queries and reserve Dense plus SPLADE for harder ones. But the gate pays for lexical retrieval before it can skip anything.

1410.14 ms

Original TREC BM25 mean query time.

1493.23 ms

Original TREC three-branch fusion mean query time.

Because the initial BM25 stage used nearly all the fusion time, skipping neural work saved little. We therefore optimized lexical scoring before assessing the extra benefit of the same frozen gate.

Indexed BM25

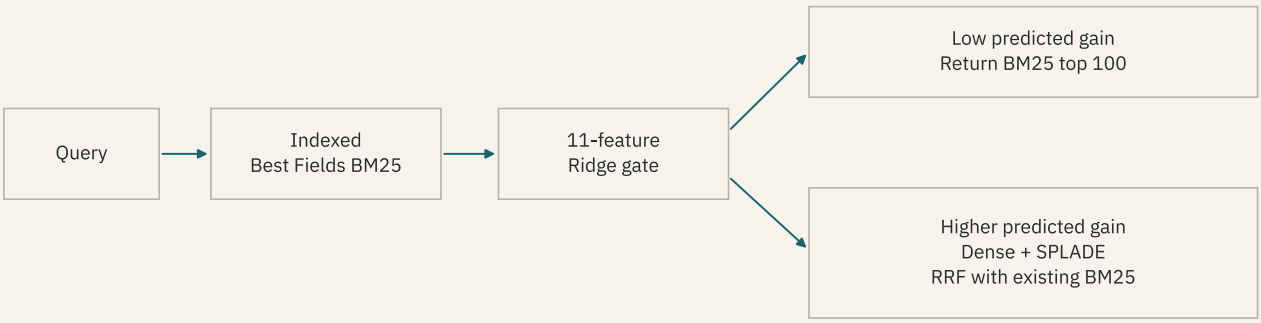
Compatible postings move reusable term contributions into the lexical index. We apply this backend equally to fusion, standalone BM25 and every cascade policy. Saved retrieval outputs remain identical.

Indexed BM25 takes 8.11 ms in the TREC follow-up.

The original/indexed comparison spans separate runs and includes session variation. Query-dependent strategy selection and eager sparse scoring have prior work. The contribution here is measured engineering and an analysis of quality/cost limits.

THE LIVE RETRIEVAL DECISION

BM25 first, then a learned gate



The fallback reuses BM25. Methods do not borrow precomputed neural query rankings from one another.

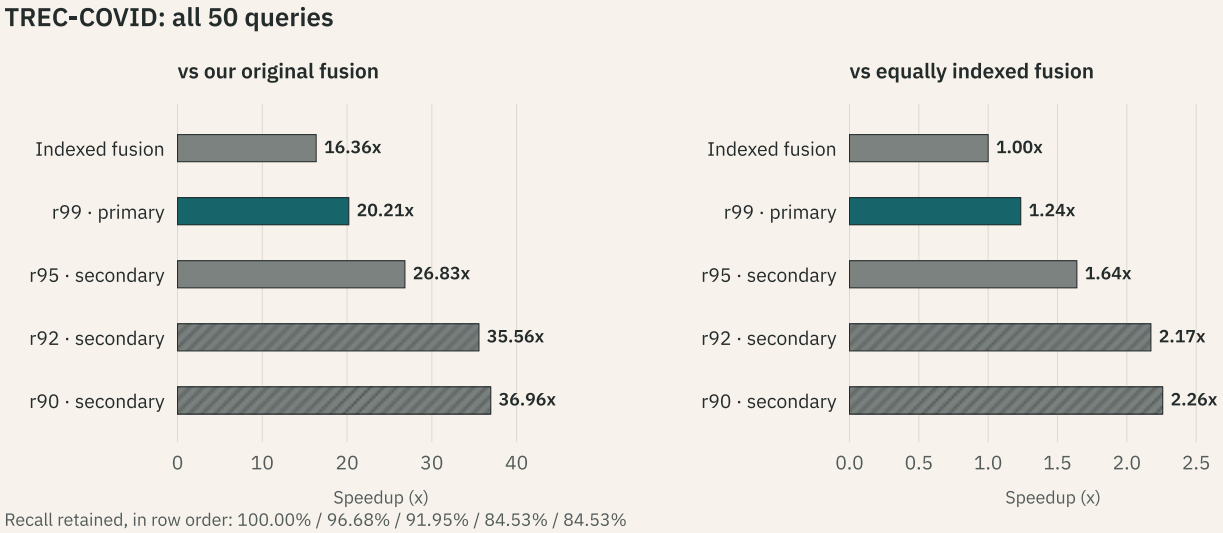
The standardized Ridge regressor predicts the clipped per-query recall gain of fusion over BM25. Five-fold out-of-fold predictions on 2,590 NFCorpus train queries calibrate four thresholds. The deployed model then fits all train queries. No evaluation relevance labels drive the runtime gate.

Compatible lexical scoring	src/indexed_bm25.py preserves tokenization, IDFs, field scoring, duplicate query-token contributions and the corpus-order tie rule.
Frozen execution	experiments/25_benchmark_indexed_cascade.py uses the existing 11-feature weights and four thresholds. RRF uses k = 60 and returns 100 documents.
Independent scoring	Experiments 29/31 reconstruct raw relevance labels. Experiment 30 checks exact old/new rankings on shared query IDs.
Recorded evidence	All 400 TREC and 2,048 sampled SQuAD first-pass rankings, scores, routes and recalls match across backends. Source and weight fingerprints accompany the reports.

The timing window includes query encoding, retrieval and routing, with GPU synchronization. Corpus encoding, index construction, model loading, downloads and answer generation remain outside it. Query-paired bootstrap intervals do not cover all between-session or hardware variation.

TREC-COVID: indexing helps, routing adds more

All 50 queries, 171,332 documents, three warmed T4 passes. All four policies keep their frozen NFCorpus-trained weights and thresholds.



Recall retained, in row order: 100.00% / 96.68% / 91.95% / 84.53% / 84.53%

Both speedup references appear. The indexing-only row precedes every gate. Hatched bars fail the 90% recall floor. Row order describes policy settings, not corpus-size scaling.

Configuration	Recall retained	Mean ms	vs indexed	vs original
Indexed fusion	100.00%	91.28	1.00x	16.36x
r99 · primary	96.68%	73.90	1.24x	20.21x
r95 · secondary	91.95%	55.66	1.64x	26.83x
r92 · secondary *	84.53%	41.99	2.17x	35.56x
r90 · secondary *	84.53%	40.41	2.26x	36.96x

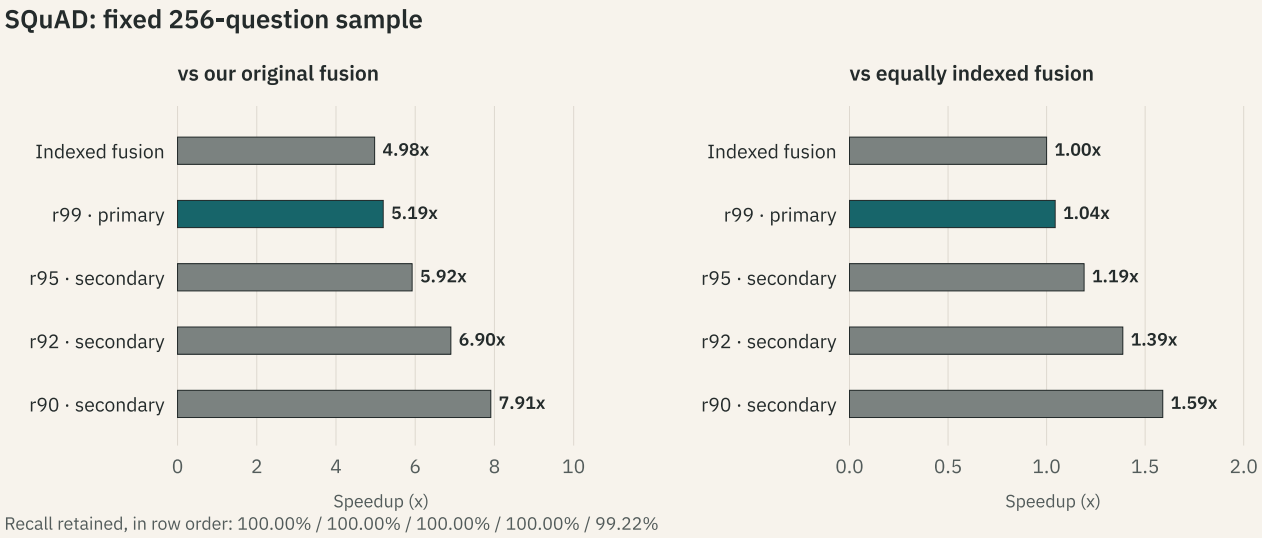
Primary r99: 96.68% retention, 95% interval 93.58–98.75%. Mean time saved: 19.04%, interval 9.47–30.11%. It meets the requested 90% floor in this setting while missing its nominal 99% calibration target.

Read the speedups against their references: 1.24x compares indexed cascade with indexed fusion and isolates routing. 16.36x compares indexed fusion with our earlier, non-indexed fusion. 20.21x compares indexed cascade with that earlier fusion, combining indexing and routing across separate runs. These ratios are within our Python implementation and are not timing comparisons with the published Elasticsearch system.

Secondary r95: 1.64× at 91.95% retention. Its interval, 86.88–95.76%, crosses below 90%. The maximum r90 point, 2.26× / 36.96×, retains only 84.53% and fails that floor. Asterisks mark failed points.

SQuAD: a declared query sample

256 fixed-hash questions, full 20,958-document corpus, three T4 passes. Original implementation evidence also covers all 10,570 dev questions with one complete timing pass.



Cross-run ratios use exactly the same 256 questions. Original sample timing comes from one complete pass, indexed timing from three. Sample quality and runtime do not establish full-query indexed performance.

Configuration	Recall retained	Mean ms	vs indexed	vs original
Indexed fusion	100.00%	29.55	1.00x	4.98x
r99 · primary	100.00%	28.31	1.04x	5.19x
r95 · secondary	100.00%	24.82	1.19x	5.92x
r92 · secondary	100.00%	21.31	1.39x	6.90x
r90 · secondary	99.22%	18.59	1.59x	7.91x

Fastest cascade, r90: 99.22% recall retained and 37.08% less mean time than indexed fusion. Its retention interval is 98.05–100.00%. Zero observed loss at the other settings is not a guarantee on unmeasured questions.

Mean gains do not ensure lower tail latency. SQuAD r90 p95 is 31.48 ms versus fusion's 31.37 ms. On TREC, primary p95 is 99.41 ms versus fusion's 97.66 ms.

The cascade is not always the best choice

TREC-COVID standalone methods

Retriever	Recall retained	Mean ms	nDCG@10	MAP@10	vs indexed	vs original
Indexed fusion	100.00%	91.28	0.7632	0.0186	1.00x	16.36x
BM25	71.38%	8.11	0.5497	0.0114	11.25x	184.02x
Dense MiniLM	69.81%	30.41	0.4725	0.0105	3.00x	49.10x
SPLADE	104.69%	52.77	0.7290	0.0166	1.73x	28.29x

SPLADE retrieves more relevant documents and takes less mean time than the primary cascade. The primary has higher MAP@10, so quality metrics express a trade-off. BM25 alone reaches 184.02× over original fusion but retains only 71.38%, below the 90% floor. This standalone speedup is separate from the cascade progression.

SQuAD standalone methods, same sample

Retriever	Recall retained	Mean ms	nDCG@10	MAP@10	vs indexed	vs original
Indexed fusion	100.00%	29.55	0.8093	0.7706	1.00x	4.98x
BM25	95.70%	1.03	0.7787	0.7504	28.59x	142.25x
Dense MiniLM	95.70%	10.82	0.6878	0.6304	2.73x	13.58x
SPLADE	98.83%	16.96	0.8544	0.8283	1.74x	8.67x

BM25 alone already exceeds a 90% recall-retention floor at 1.03 ms. SPLADE supplies the strongest sampled nDCG/MAP. Fusion-relative savings do not establish the cascade's overall superiority.

Earlier evidence remains visible

NFCorpus dev primary r92 retained 94.74% with 23.00% mean savings. SCIDOCS r92 retained 90.01% with 5.53% savings and an interval crossing below 90%. Every nominal calibration target failed on SCIDOCS and TREC-COVID. Those outcomes remain part of the study.

What the evidence supports

The compatible backend makes conditional neural skipping save more mean time against equally indexed fusion. Frozen policies expose a measurable speed/recall trade-off, alongside transfer failures and stronger constant alternatives.

The defensible headline is 1.24× at 96.68% retained recall on TREC-COVID.

The larger 26.83× combined point retains 91.95% and includes indexing plus routing across separate runs. Its interval crosses below 90%. The absolute maximum cascade point, 36.96×, retains 84.53% and fails the requested floor.

Remaining evaluation

Full NQ, HotpotQA and CoQA have no completed current evaluation. Earlier NQ work is exploratory. Generated-answer EM/F1, modern adaptive baselines, production concurrency, energy and index memory/update costs remain future work.

Further tests can extend this frozen release under a new declared protocol. They should preserve the strongest standalone references and distinguish additional dataset coverage from evidence of a new algorithm.

Selected primary sources

1. Sawarkar, Mangal and Solanki (2024). Blended RAG, arXiv:2404.07220v2.
2. Arabzadeh, Yan and Clarke (2021). Dense vs. sparse retrieval strategy selection.
3. Lu (2024). BM25S: eager sparse scoring.
4. Thakur et al. (2021). BEIR.
5. Rajpurkar et al. (2016). SQuAD.
6. Formal, Piwowarski and Clinchant (2021). SPLADE; Reimers and Gurevych (2019). Sentence-BERT.

Completed experimental release, 21 October 2026. The research paper contains full methods, tables, limitations and references. Figures and tables use saved project reports. This showcase works offline with embedded fonts and charts.

Read the paper · Presentation PDF